Project Proposal

Trading Journal: Leveraging MCP in the Financial Domain

Revised proposal with related work, preliminary prototype evidence, project coordination plan, and application flowcharts.

Project thesis. Trading Journal is proposed as a Python-based portfolio and trade-journaling application that also serves as a practical case study for Model Context Protocol client-server design in a financial software setting.

Introduction

Model Context Protocol (MCP) is an open standard for connecting AI applications to external systems, including data sources, tools, and reusable workflows. The official MCP documentation describes MCP as a standard that lets applications such as Claude or ChatGPT connect to external files, databases, tools, and prompts through a consistent interface. In practical terms, MCP gives an AI-capable application a way to discover what a system can do, call structured tools, read resources, and use reusable prompts without building a separate custom integration for every data source.

This project proposes the development of Trading Journal, a Python-based portfolio tracking and trade-journaling application. The application is inspired by the day-to-day usability of retail brokerage portfolio views, but it adds a journaling layer that records the reason for each trade. The project has two connected goals. The first goal is technical and academic, to learn and demonstrate MCP by building a real application that exposes MCP servers and also consumes MCP capabilities as a client. The second goal is to build a useful local-first application for tracking holdings, entering trades, reviewing profit and loss, and analyzing investment decisions.

The preliminary prototype already demonstrates the critical path for the project. It supports manual holding import, manual trade entry, realized and unrealized profit/loss calculations, live market-data integration for supported instruments, two MCP servers, and an external command-line MCP client. This makes the proposal stronger than a purely conceptual plan because the most important architecture choices have already been tested in a working system.

Summary of the Proposal

The proposed system is a full-stack Python application for portfolio management, trade journaling, and MCP-based interoperability. In its first complete version, the application will allow users to import current holdings manually, record new buy and sell trades, close positions, and review performance across open and closed positions. Every trade will require a reason field so the user can later analyze not only what happened financially, but also why the decision was made.

The project will also serve as a learning oriented MCP implementation. The Trading Journal application will expose portfolio and journaling features through a Trading Journal MCP server. Market data features will be exposed through a separate Market Data MCP server. A command-line MCP client has already been created to demonstrate how an outside client can discover and use those capabilities. This gives the project a clear academic contribution: it shows MCP in a realistic domain rather than as a small artificial example.

The final project will balance application development with a high quality submitted article. The article will discuss the architecture, related work, security considerations, implementation decisions, and lessons learned from building a financial MCP application. The application and article together form a reasonable two person project because both team members will contribute to both facets, with specific ownership rotating by milestone rather than by a strict developer/writer split.

Related Work

The official MCP documentation frames MCP as an open protocol for connecting AI applications to external systems. This is directly relevant to Trading Journal because the project treats the portfolio application as a provider of structured capabilities rather than only as a web interface. The application exposes tools such as portfolio summary retrieval and trade entry, resources such as position snapshots, and prompts such as daily portfolio reviews. This follows the core MCP model of making external context and capabilities available to AI systems through a standard interface.

The MCP specification describes the protocol as a standardized way for applications to share contextual information, expose tools, and build composable workflows using JSON-RPC 2.0, capability negotiation, resources, prompts, and tools. The specification is important for this project because it provides the conceptual boundary between a normal REST API and an MCP-enabled application. In Trading Journal, the MCP layer is not only another endpoint but also a capability surface that can be discovered and used by external clients.

The 2026 MCP roadmap identifies several areas that are especially relevant to this project: transport scalability, agent communication, governance maturation, enterprise readiness, stronger metadata discovery, and deeper security and authorization work. The current prototype is intentionally local-first, but the roadmap helps shape the future direction of the project. For example, the prototype uses streamable HTTP locally, while the full version will need to consider authentication, deployment, authorization, and remote-server readiness before it can safely be used over the internet.

The MCP landscape and security review recommended by Dr. Batthula is especially important because financial applications involve sensitive user data and potentially high-impact actions. That work highlights risks across the MCP server lifecycle, including malicious or misleading tools, sandbox escape risks, configuration drift, privilege persistence, lack of centralized oversight, and authentication gaps. These concerns directly influence this proposal. The full application should not expose unrestricted trade or account actions to remote clients without authentication, user consent, careful logging, and least-privilege tool design.

The Decode the Future overview provides a useful applied explanation of MCP primitives: tools act, resources provide read-only context, and prompts standardize reusable workflows. This distinction maps cleanly onto the Trading Journal prototype. Adding a manual trade is a tool because it changes application state. Reading portfolio positions is a resource because it retrieves data without changing it. Generating a portfolio review instruction is a prompt because it gives an AI assistant a reusable analysis template.

FastAPI is used as the application foundation because it is a modern Python web framework designed for API development with type hints, automatic documentation, and production-oriented features. SQLAlchemy supports the database model and service-layer persistence. Alpaca is used as the preliminary market-data provider because its Market Data API supports HTTP and WebSocket access to historical and real-time market data, while also making clear that feed coverage depends on subscription level and asset class [10]. These choices keep the prototype practical while preserving room for future provider or broker integration.

Proposed Application Scope

The full application will focus on the intersection of portfolio tracking, trade journaling, and MCP-based integration. The intended first complete version will remain scoped enough for a two-person team, while still being substantial enough to demonstrate meaningful software engineering and research value.

- Support manual import of existing holdings and manual entry of buy and sell trades.
- Track open and closed positions using average-cost accounting.

- Calculate realized P&L, unrealized P&L, market value, cost basis, and P&L percentages.
- Report performance overall, by account, and by account plus asset class.
- Require a reason for each trade so the application functions as both a tracker and a decision journal.
- Use external market data for supported stocks, ETFs, and options, while clearly documenting limitations for mutual funds, bonds, and options entitlements.
- Expose portfolio capabilities through a Trading Journal MCP server.
- Expose market-data capabilities through a separate Market Data MCP server.
- Provide an external command-line MCP client and later a browser-based MCP inspection console.
- Prepare the application for later deployment with authentication, authorization, and safer remote access.

Phase	Primary Deliverables	Feasibility Rationale
Prototype validation	Manual holdings, trade entry, P&L, two MCP servers, CLI client.	Already completed as preliminary work.
Core application	Improved UI, richer analytics, broker sync foundation, secure local config.	Builds directly on the working service layer and tests.
MCP maturity	MCP console, authenticated remote access plan, stronger prompt/resource design.	Incremental extension of existing MCP endpoints.
Research article	Related work, architecture analysis, security discussion, lessons learned.	Uses the prototype as evidence and case study material.

Completed Preliminary Work

A working prototype has already been developed to validate the main technical direction of the project. The prototype is built using Python, FastAPI, SQLAlchemy, SQLite, Jinja templates, and MCP. The purpose of this early work was not to complete the production application, but to prove that the core ideas are feasible: portfolio tracking, trade journaling, profit/loss calculation, live market-data integration, and MCP-based communication between clients and servers.

The prototype currently runs as a local-first web application. It includes a database-backed portfolio model with accounts, instruments, trades, and positions. Two default account types have been created: an IBKR Live account to represent a future broker-connected account and a Manual Fidelity account to support manual import of holdings. This allows the prototype to support the initial use case where existing Fidelity holdings can be entered manually before automated brokerage synchronization is added later.

A manual holding import workflow has been implemented. This allows a user to enter existing positions into the system with account, symbol, asset class, quantity, average cost, opening date, description, and

notes. After the initial holdings are entered, the user can maintain the portfolio by manually entering new buy and sell trades. The trade-entry workflow includes account, symbol, asset class, trade date, side, quantity, price, fees, currency, notes, and a required reason for trade.

Position tracking and P&L calculations have also been implemented. The application maintains open positions using average-cost accounting. Buy trades increase position quantity and update average cost. Sell trades reduce quantity and calculate realized P&L. If a sell trade closes the full position, the position is moved from open positions to closed position history. Unrealized P&L is calculated for open positions using current market prices.

The current web interface includes pages for the dashboard, trades, open positions, closed positions, opening holding import, manual trade entry, and market data. The dashboard reports overall performance, account-level performance, and account plus asset-class performance. The market-data page displays quote information for open and closed positions where supported. The positions page includes a quote refresh workflow that updates open-position marks through MCP.

The preliminary work also includes two MCP servers. The Trading Journal MCP server exposes tools for portfolio summary retrieval, account listing, position listing, trade listing, opening holding creation, and manual trade creation. It also exposes resources such as portfolio://summary and portfolio://positions, and prompts such as daily_portfolio_review and journal_follow_up. The Market Data MCP server exposes tools for provider capability checks, equity snapshots, and option snapshots, along with the market-data://capabilities resource and a market_data_health_check prompt.

The application also acts as an MCP client internally. When the user refreshes market prices, the Trading Journal application starts an MCP client session and calls the Market Data MCP server rather than directly embedding market-data calls in the page route. This demonstrates both sides of MCP in one system: the application exposes capabilities as a server and consumes capabilities as a client.

A separate command-line MCP client has been created to support demonstration and evaluation. This client can explain MCP concepts, list available MCP servers, discover tools/resources/prompts, call tools, read resources, render prompts, and run guided workflows such as portfolio-review, market-check, and client-demo. This is important because it proves that another process can use the application's MCP capabilities without using the browser interface or importing the application source code.

Area	Current Prototype	Planned Full Application
Portfolio model	Accounts, instruments, trades, positions, open and closed position states.	Broker-aware data model with sync history and production audit metadata.

Trade journaling	Manual trades with required reason, notes, fees, and realized P&L on sells.	Richer review workflow, tagging, attachments, and post-trade reflection prompts.
Market data	Alpaca snapshots for supported stocks, ETFs, and options through Market Data MCP.	Provider hardening, entitlement handling, fallback pricing, and clearer asset-class coverage.
MCP capability	Two MCP servers, resources, prompts, internal MCP client, and external CLI client.	Authenticated remote MCP access, browser MCP console, and stronger consent controls.
Testing	Automated tests for trade flow, P&L, dashboard summaries, and MCP behavior.	Expanded integration tests, security checks, and deployment smoke tests.

Application Flowcharts

The following figures summarize the current prototype and proposed full application architecture. They are included to make the project easier to review visually and to clarify how MCP fits into the application.

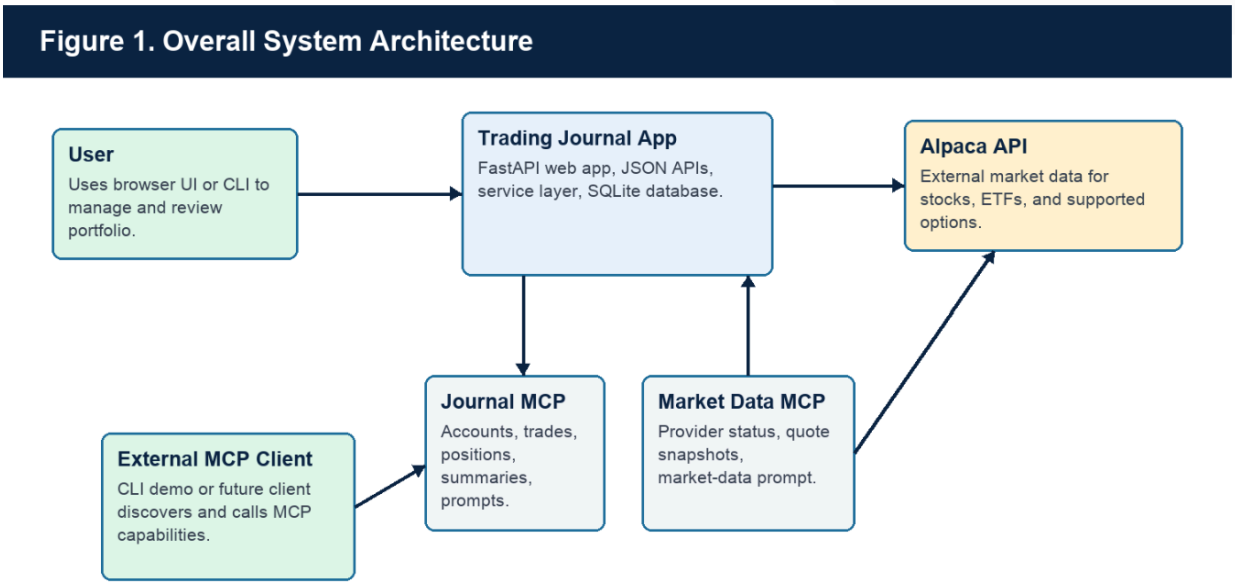

Figure 1. The application combines a local FastAPI web app, database-backed portfolio logic, two MCP servers, an external MCP client, and an external market-data provider.

Figure 2. Day-to-Day User Workflow

Figure 2. Day-to-day usage begins with importing holdings, then entering trades, refreshing quotes, reviewing P&L, and analyzing journal decisions.

Figure 3. Trade and P&L Lifecycle

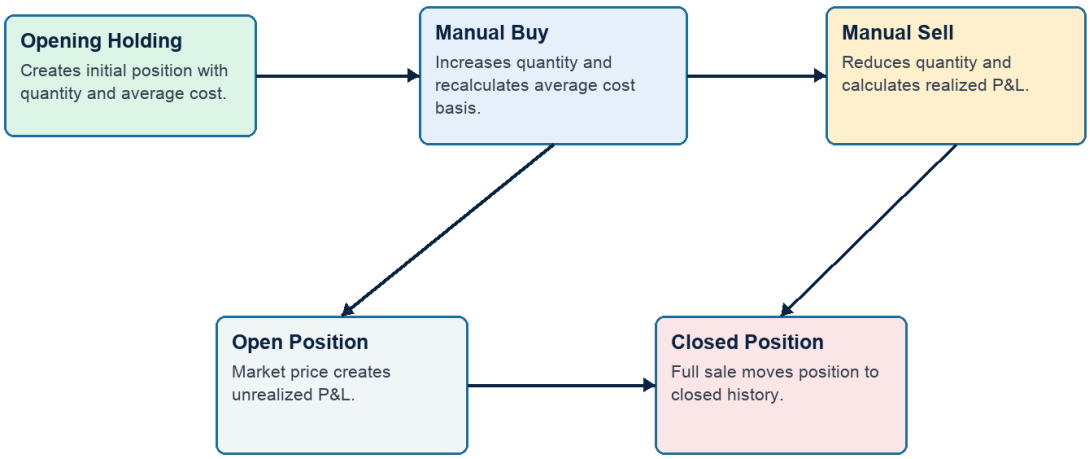

Figure 3. Trade activity updates positions and moves fully sold positions into closed history while preserving realized P&L.

Figure 4. MCP Client-Server Interaction

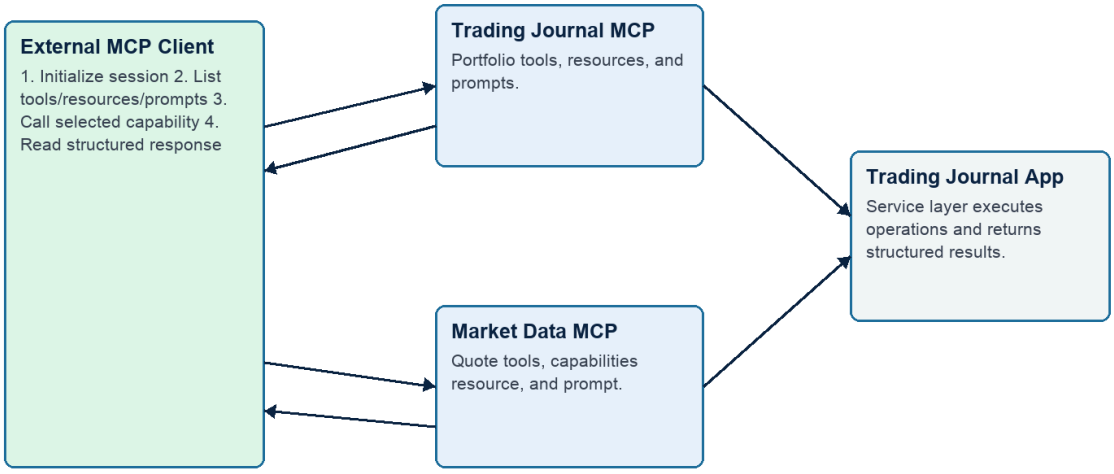

Figure 4. An external MCP client discovers and uses tools, resources, and prompts from either MCP server.

Figure 5. Market Data Refresh Flow

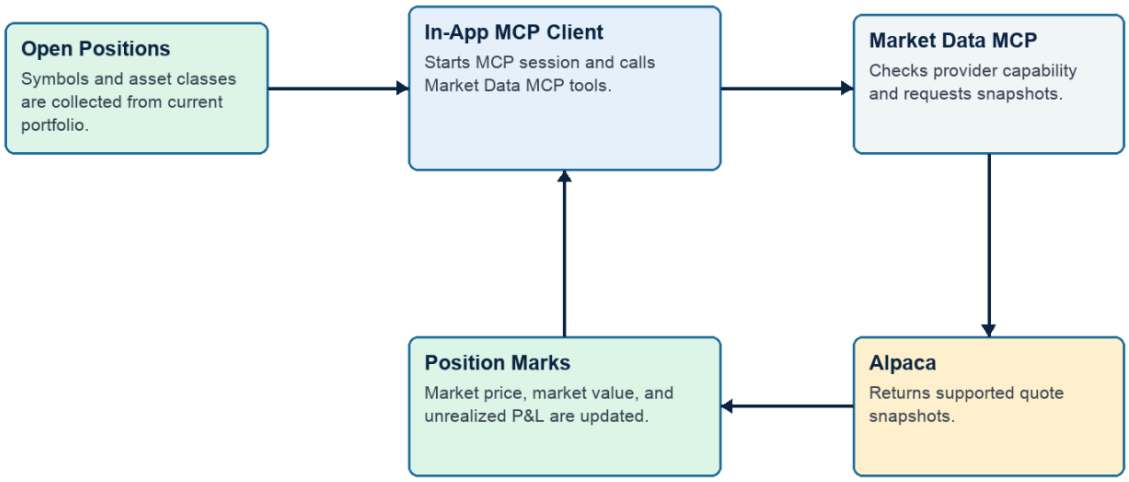

Figure 5. Market-data refresh uses an in-app MCP client to call the Market Data MCP server and update open-position marks.

Figure 6. Proposed Two-Person Work Plan

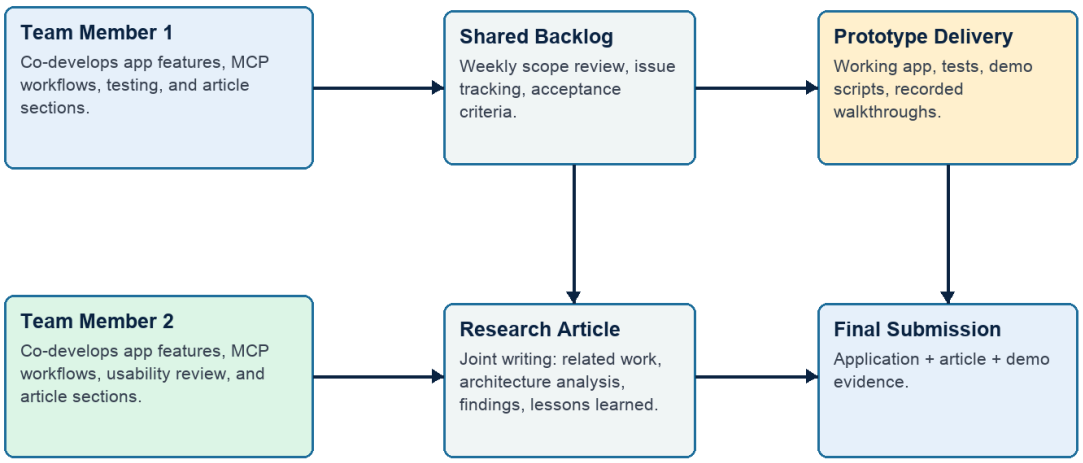

Figure 6. A two-person structure keeps both members involved in implementation and research/article work while maintaining a shared backlog and final integrated deliverable.

Project Team

This project will be completed by a two person team. Both team members will contribute to the technical implementation and the research/article components of the project. The responsibilities are intentionally divided so each team member develops hands-on experience with MCP, application design, financial domain workflows, testing, documentation, and research communication.

- Team Member 1: Deepak Singla. Co-develop the Trading Journal application, implement and test MCP server/client workflows, contribute to portfolio and market-data features, participate in related-work analysis, and help write the final article.
- Team Member 2: Ramkiran Hota. Co-develop application features, assist with MCP integration and testing, contribute to UI/workflow design, participate in related-work analysis, and help write the final article.
- Shared responsibilities: weekly planning, implementation reviews, prototype testing, documentation, demo preparation, architecture diagrams, security discussion, and final submission.

Project Plan and Scope Justification

The project is reasonable for a two person team because the preliminary prototype has already reduced the highest technical uncertainty. The remaining work is not starting from zero as it is an extension and hardening of an existing codebase. The work will not be split into separate technical and writing roles. Instead, both team members will contribute to both the application and the article, with individual task ownership rotating by milestone.

For example, one milestone may have Team Member 1 leading a backend MCP feature while Team Member 2 leads the matching UI/demo documentation. In a later milestone, those roles can rotate so both members gain experience across implementation, testing, usability, related work, and article writing. Both members will review each other's work through a shared backlog, weekly check-ins, and milestone-based demos.

The project scope is justified because the final deliverable is not only a production-ready trading platform. It is a technical project plus a high-quality submitted article. That means the team can focus on a strong, demonstrable subset of features: reliable manual portfolio tracking, correct P&L logic, meaningful MCP integration, clear market-data boundaries, and a well-written analysis of the architecture and lessons learned. More advanced features such as full internet deployment, automated broker trading, and production-grade multi-user support can be treated as future work unless time permits.

- Weekly coordination: one planning meeting, one demo/checkpoint, and shared issue tracking.
- Rotating technical ownership: MCP architecture, backend services, data model, broker-sync foundation, automated tests.
- Rotating research and UX ownership: related work, proposal/article writing, diagrams, usability review, demo script, documentation.
- Shared responsibilities: scope decisions, security review, final integration, presentation, and submitted article quality.

Security and Ethics Considerations

Security is central to this project because the financial domain involves sensitive account data and potentially high-impact actions. The prototype is intentionally local-first, and the current MCP endpoints are intended for local development and demonstration. A full internet-deployed version must add authentication, authorization, transport security, logging, and explicit user-consent flows before exposing MCP capabilities remotely.

The related work on MCP security identifies risks such as tool poisoning, ambiguous tool names, sandbox escape, configuration drift, and authorization gaps. The project will address these risks by keeping tool names explicit, separating read-only resources from write-capable tools, documenting tool side effects, avoiding automatic execution of sensitive actions, and designing future remote access around least privilege and user approval. In a financial application, MCP tools that change portfolio state must be treated differently from resources that only read summary data.

The project is intended for journaling and analysis, not for automated trading execution in the first version. This boundary reduces risk and keeps the application aligned with the academic objective: to study MCP architecture and build a useful portfolio journal. Any future integration with live brokerage trading would require additional compliance, risk, and user-confirmation controls.

Evaluation Plan

The project will be evaluated through both technical and usability evidence. Technical evaluation will focus on correctness of trade workflows, P&L calculations, market-data behavior, MCP discovery, MCP tool calls, resource reads, prompt rendering, and regression tests. Usability evaluation will focus on whether the application supports the intended daily workflow: importing holdings, entering trades, recording trade reasons, closing positions, reviewing performance, and demonstrating MCP behavior clearly.

- Functional tests for trade entry, holding import, P&L calculations, and closed-position behavior.
- MCP tests for tool discovery, resource access, prompt rendering, and multi-server workflows.
- Demo scripts showing CLI-based external client behavior.
- Manual usability checks for dashboard, positions, trades, market data, and journaling workflows.
- Article-based evaluation discussing architecture tradeoffs, related work, and future improvements.

Planned Next Steps

1. Complete the browser-based MCP console so MCP tools, resources, and prompts can be inspected without relying only on terminal commands.
2. Improve UI polish and navigation for daily portfolio use.
3. Expand market-data handling, including clearer unsupported-asset messages and provider configuration guidance.
4. Prepare an IBKR-first synchronization design while keeping manual Fidelity import available.
5. Add authentication and deployment planning for future internet access.
6. Develop the submitted article using the prototype as a case study in MCP architecture for financial applications.

Conclusion

Trading Journal is a feasible and meaningful project because it combines a practical user-facing application with a current technical research topic. The preliminary prototype already proves the central idea: a portfolio journal can expose useful capabilities through MCP, consume market-data capabilities through another MCP server, and be demonstrated by an external MCP client. The proposed next phase will mature this prototype into a stronger application and a well-supported article that discusses MCP's role, benefits, limitations, and security considerations in the financial domain.

References

[1] Model Context Protocol. (2026). What is the Model Context Protocol (MCP)?

https://modelcontextprotocol.io/docs/getting-started/intro

[2] Model Context Protocol. (2025). Specification, 2025-06-18.

https://modelcontextprotocol.io/specification/2025-06-18

[3] Soria Parra, D. (2026). The 2026 MCP Roadmap. Model Context Protocol Blog.

https://blog.modelcontextprotocol.io/posts/2026-mcp-roadmap/

[4] Eleventh Hour Enthusiast. (2025). Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions.

https://medium.com/@EleventhHourEnthusiast/model-context-protocol-mcp-landscape-security-threats-and-future-research-directions-488b8d2eade8

[5] Decode the Future. (2026). What Is MCP? Model Context Protocol Explained for 2026.

https://decodethefuture.org/en/what-is-mcp-model-context-protocol/

[6] FastMCP Documentation. Getting Started.

https://gofastmcp.com/getting-started/welcome

[7] FastAPI Documentation.

https://fastapi.tiangolo.com/

[8] SQLAlchemy Documentation.

https://docs.sqlalchemy.org/

[9] Alpaca Documentation. Getting Started.

https://docs.alpaca.markets/us/docs/getting-started

[10] Alpaca Documentation About Market Data API.

https://docs.alpaca.markets/us/docs/about-market-data-api

[11] Alpaca Market Data Overview. https://alpaca.markets/data